

Chapter 3

CS1 course package - Knox College (KC)

David P. Bunde¹ and Jaime Spacco²

¹Knox College, dbunde@knox.edu

²Knox College, jspacco@knox.edu

Abstract

Knox College hosted the Java-based development teams in this project. Its CS1 course was updated with three new activities: Unplugged Flag Maker Activity, Knoxcraft, and using Greenfoot as a game engine. These activities introduce the ideas of parallel execution and dependencies, give students experience interacting with a remote server, and has them write event-driven code. Although these activities cover this new material, the focus of what students actually do is on traditional topics such as loops and object state so that they emerge from the class with the skills their peers had in previous terms, but with greater exposure to modern topics.

Descriptor Elements

Textbook used: None

Programming Language Employed: Java

Relevant core courses: CS1

Pre-requisites: None

Relevant PDC topics: Classification of PDC Topics according to Bloom's Taxonomy[1]: Remember (RE), Understand (UN), Apply (AP), Analyze (AN), Evaluate (EV), and Create (CR)

PDC Concept	Chapter Section				
	1.1	1.2	1.3	1.4	1.5
Speedup	?	?	?		
Parallel problem decomposition	?	?			
Data parallelism	?	?			
Dependencies	?	?			
Distributed computing	?	?			
Event handling		?	?		

Learning outcomes: After this course, in addition to the existing learning outcomes from the previous version of the courses, students are able to do the following:

1. define and calculate speedup,
2. give examples of data parallel computing,
3. explain dependencies and give examples when they occur,
4. give examples when a remote server is useful for a program, and
5. write simple programs that use event handling.

Context for use: Knox College is a small, residential liberal arts institution. The academic calendar has 3 terms (Fall, Winter, and Spring), each slightly less than 10 weeks long. The CS1 class meets 4 times a week, 3 lectures (MWF) and one lab meeting per week. All class meetings are 70 minutes long. The intent is that a Knox course should have equivalent class time to a semester course at other institutions and cover the same amount of content.

The size of this class varies, but the targeted size is 24 students. The class is the first course in the Computer Science major, the Computer Science minor, and the Data Science major. It is also required for students majoring in Biology and Neuroscience who complete a BS instead of a BA in those fields. The course is also a common option for students in the Business & Management major and is taken by many students as an elective. Historically, roughly half of the students from this course continue to CS2.

This course uses Peer Instruction [4, 5], a pedagogy where lecture is periodically interrupted with a multiple-choice question. The students answer these individually, then discuss them in groups and re-answer. Then the instructor reveals the correct answer, has a student explain this answer, and then fills in any additional material. The in-class questions are graded only for participation. Lab assignments are homework; the students begin the assignments in lab, when the instructor and a TA are available to answer questions. The in class work gets them over the hurdle of starting the assignment. Then they have until the beginning of the next lab (a week) to complete the assignment.

3.1 Introduction

As one of the development teams, Knox introduced three significant changes to its CS1 course (Flag Maker, Knoxcraft, and Greenfoot) to introduce data parallelism, distributed computing, and event handling, the three topics identified as suitable for CS1 early in the project [2]. We also preserved other aspects of our modernization goals; the course already included graphical assignments (which we increased in number) and used modern pedagogy (Peer Instruction [4, 5], which we retained).

3.2 How CS1 was changed and PDC topics integrated

The time for the new material came out of a previously-assigned 2-week Media Computation lab in which students implement image processing operations by manipulating image pixels. (It was called Photoshop Jr.) The focus of this assignment was on loops, particularly nested loops. The course also covered nested loops in the programming part of Plugged-In Flag Maker Activity, which was already included. Thus, we added the unplugged and parallel parts of Flag Maker (roughly 1 lecture day). One of the weeks was then devoted to Knoxcraft, which also covers loops and nested loops. The other week was used to expand the existing unit on object-oriented programming into the Plugged-In Greenfoot Activity, which also includes this material (in a more compelling way) and adds exposure to remote servers and distributed computing.

3.2.1 Course calendar

Here is the new course calendar at a week-by-week level:

Week	Topics
1	Course introduction; types and variables; for loops; calling constructors and methods
2	Conditionals; booleans; loops
3	Strings; nested loops; data parallelism
4	Writing functions (static methods); return values
5	Arrays; ArrayList; identifying mystery functions
6	Effective comments; using a server
7	Writing classes; event-driven programming
8	RPC; more practice
9	Recursion

Day-by-day level detail as well as the specific lecture materials and assignments are provided in the online materials accompanying this chapter, as detailed in the chapter's appendix.

3.2.2 Challenges

SS: Discuss logistical, pedagogical, or technical difficulties

3.3 New Activities

3.3.1 Flag Maker

Activity Summary

This activity uses the task of coloring the pixels of a flag to motivate students to practice for loops and then unplugged activities to introduce parallel concepts. It builds on the previously-described Plugged-In Flag Maker Activity (Section 1.4.1) and Unplugged Flag Maker Activity (Section 1.3.1) activities. After the unplugged activity, we then added some explicit discussion of parallel task decomposition and dependencies. The entire activity is described by Smith et al. [6]; some of the material here is taken from that description.

Prerequisites

We used this activity early in the term (week 3) so the prerequisites are minimal. Students do need to be able to write for loops and call methods for the programming aspects of the assignment. (They do get to follow a sample implementation.) The prerequisites for the unplugged components are even weaker; students need to have an understanding of the step-by-step nature of code. (Some of the slides after the unplugged activity explicitly reinforce students' ability to trace nested loops.)

Learning Outcomes

Implementation Details

Most of what we did for this activity is described elsewhere. In week 3 of the course, the students had already been introduced to integer variables and operations, boolean variables and conditional expressions, calling constructors and methods, and `for` loops. Then they are given the Plugged-In Flag Maker Activity in lab and told to complete it as homework. The next day, we do the Unplugged Flag Maker Activity.

At the end of that same period, we go through a deck of slides. The first couple of these reinforce loop concepts (which cell is visited first in a traversal by nested loop and which cell is visited second). Then the slides introduced the idea of dependencies. The coloring pixels in parallel is in tension with an important technique for more complicated flags: coloring different elements of the flag in layers. For example, the flag of Great Britain (Figure 3.1) is most easily made by coloring the entire flag blue, then adding the crossing diagonal white lines, and then finally coloring the red vertical and horizontal lines. This approach avoids having to make complicated intersection tests between the flag's different features. (The idea is the same as the Painter's algorithm in 3D graphics, which renders complex scenes by drawing polygons in the order of their distance from the camera.) Unfortunately, this approach also limits parallelism by introducing dependencies: the background must be colored before the diagonals, which must be colored before the rectilinear lines. Since the students had been working on more complicated flags as part of the flag coloring programming assignment, they readily identified this issue and the difference between parallelizing the flag coloring for Mauritius and Great Britain.

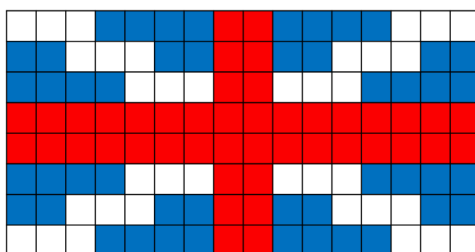


Figure 3.1: Flag coloring assignment version of the flag of Great Britain

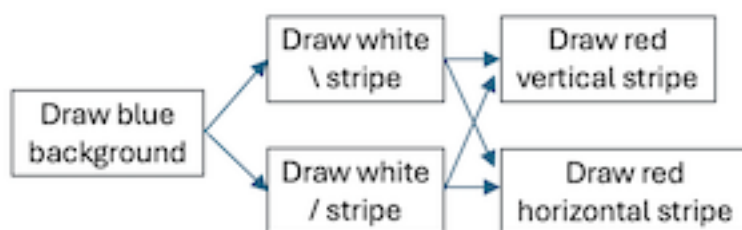


Figure 3.2: Dependency graph for coloring the flag of Great Britain

To formalize this idea, the students were given the definition of a dependency graph (vertices are tasks and directed edges denote dependencies) and shown the example for coloring the flag of Great Britain shown in Figure 3.2. Then, to show their understanding, the students are asked to draw one for coloring the flag of Jordan (Figure 3.3).

Instructional Material

Materials for all these activities are in the repo corresponding to this section: https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter3-Knox/Unplugged_activities/flagmaker. This includes the plugged assignment materials, instructions and slides for the unplugged activity, and then the additional slides after the unplugged activity.

Another resource that might be useful is an explanation of data parallelism using parallel pixel coloring: <https://www.youtube.com/watch?v=WmW6SD-EHVV>. It's explained as the difference between graphics with a CPU (one pixel per cycle) and a GPU (all pixels in a cycle). This is suggested as a possible pre-class viewing for students.

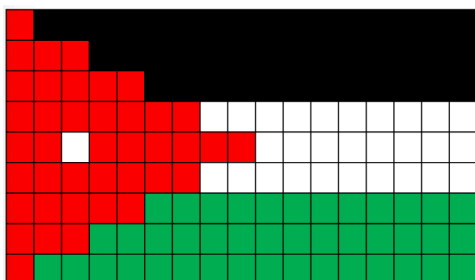


Figure 3.3: Flag coloring assignment version of the flag of Jordan

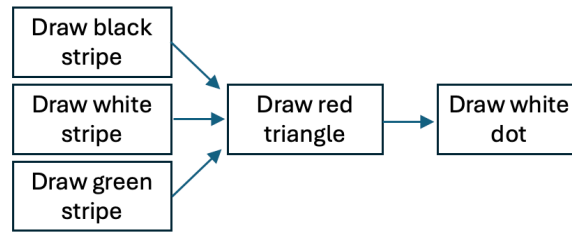


Figure 3.4: Dependency graph for coloring the flag of Jordan

Experience and Teaching Tips

Data and Analysis

In Fall 2024, we first used this activity in a CS1 class of 65 students split into three sections. To evaluate the activity, we collected the dependency graphs that students drew for coloring the flag of Jordan (see Figure 3.3). These were collected anonymously; the students were told that submission was voluntary and would have no impact on their grade. A total of 29 drawings (45% response rate). The first section is underrepresented in the count (only 4 drawings submitted), likely because the other activities ran long and they had less time to draw.

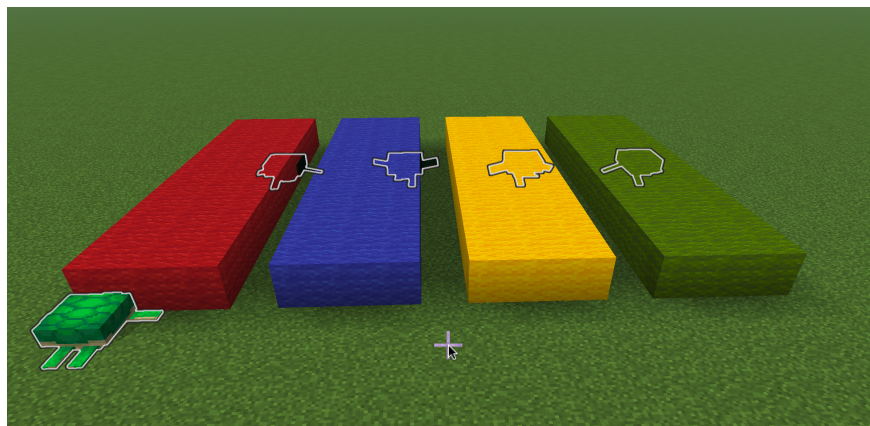
We examined the submissions to evaluate the level of understanding demonstrated. Our intended solution for the problem is shown in Figure 3.4; the stripes form the first layer and must be drawn first, followed by the red triangle, and then the white dot (a star in the actual flag). We accepted solutions that omitted the box for drawing the white stripe; in the Plugged-In Flag Maker Activity, the background is initially white so students had been “creating” white features by leaving that area blank.

The students who were at least mostly correct made up 59% of the respondents. Of the submissions, 10 (34%) were perfectly correct. Seven (24%) more were mostly correct. Five of these split the red triangle into upper and lower parts, which was again consistent with how some had been drawing such features in Plugged-In Flag Maker Activity, but which requires a more complicated dependency graph since the two halves have different dependencies. One of the other two mostly-correct submissions used one task for all the stripes and the other suggested the dependencies spatially but omitted the arrows.

Of the students who were not at least mostly correct, the most common error was to draw a linear chain of tasks. This suggests that they either thought about the graph in terms of sequential code or misunderstood the meaning of a dependency. Other submissions were incomplete but seemed to be working toward a linear solution and four (14%) students did not demonstrate any understanding; they drew the flag or started giving code to draw it.

Although we are reasonably happy with the level of understanding demonstrated, we plan to either spend more time on dependencies or include them in other assignments to reinforce this concept.

Figure 3.5: Parallel Knoxcraft Turtles.



3.3.2 Knoxcraft

Activity Summary

This activity is inspired by Papert's Logo [3], where a 2D turtle walks around a 2D plane drawing pictures. Knoxcraft uses the voxel based world of Minecraft as a 3D canvas for a 3D version of Logo. Students write code in their IDE (VS Code, NetBeans, etc), call a special method to upload to the server, and then log into a specially modded Minecraft server to observe a turtle (literally a Minecraft turtle the Knoxcraft mod hijacks for this purpose).

Prerequisites

Knoxcraft requires the ability to call constructors and methods. Interesting creations will require loops and possibly conditionals. Recursion is a bonus, allowing the creation of recursive structures. The assignment involves connecting to a remote server, but the code for this is provided so students don't need to understand those calls. Students write all of their code in their IDE, and call an upload functions to upload to the server. The server doesn't actually run their code; it runs client side and produces a series of instructions in JSON for the server-side turtles to follow.

Learning Outcomes

Knoxcraft enables parallelism. Students can create multiple "threads" of execution, which will map to different physical turtles. Figure 3.5 shows multiple Knoxcraft turtles building the flag of Mauritius in parallel. Multiple Knoxcraft "threads" build structures observably faster, giving students a visual demonstration of the value of parallelism.

Implementation Details

There are two main challenges for Knoxcraft:

- Faculty need to run a specially modded Minecraft server. This is not difficult, but it does require a bit of time and effort.

- Students need access to Minecraft accounts. Students can run programs uploaded by other students, so not every student needs a Minecraft account.

Instructional Material

Materials for all these activities are in the repo corresponding to this section: https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter3-Knox/Plugged_activities/knoxcraft. This includes the background material and lab assignments.

Experience and Teaching Tips

- Give an example 3D structure, and require at least one 3D structure. One of our worked examples is a 2D flag, similar to what they have already done with the Flagmaker assignment, and unless we said otherwise, many students simply swapped in a different flag, skipping the 3D aspects of the project.
- Give a large structure with no threads, and a large structure with threads. Run both and have students time them.
- Require a threads if you want students to use them.
- Collect and share a "gallery" of images of the most interesting creations.

Data and Analysis

We did not collect data specifically on this assignment.

3.3.3 Greenfoot

Activity Summary

Greenfoot is described in detail in Section 1.4.2. It is a fun and intuitive 2D environment for making games or simulations that is accessible to novice Java programmers.

Prerequisites

Greenfoot uses Java, though faculty teaching C++ at Casper College taught their students Greenfoot. Students need to understand methods, instance variables, classes, and loops. Knowledge of inheritance is helpful, but the use of subclasses is simple enough that CS-1 students can use Greenfoot without understanding subclasses. We teach Greenfoot in place of the (extremely boring) material we used to teach on object-oriented programming.

Learning Outcomes

Callbacks and event-driven programming. Greenfoot provides a mechanism to explicitly order the callbacks; otherwise the ordering is undetermined and can lead to unpredictable results.

Implementation Details

Greenfoot is available for Mac, Windows, or Linux, and is freely available for download and use.

Instructional Material

Materials for all these activities are in the repo corresponding to this section: https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter3-Knox/Plugged_activities/greenfoot. This includes the background material and both lab assignments, plus videos and "worked example" sample code to share with the students about creating games in Greenfoot.

Experience and Teaching Tips

- Give students open-ended assignments with a minimum set of requirements, but no maximum. Trust in the creativity of students to take ownership of their projects.
- Share "worked examples" with the students, such as the four sample projects provided in the appendix.
- Create a "gallery" of the most interesting submissions to share with the class, or to show the next year's class. Set high expectations and motivate students with their desire to perform well in front of their peers, not pressure to achieve grades.
- Disallow games with many examples already shared and uploaded to <http://greenfoot.org>, such as the snake game and flappy bird.
- Ask students to make a short video explaining and demonstrating the best features of their game. This helps build presentation skills, and videos can be shown to future classes.

Data and Analysis

We did not collect data specifically on this assignment.

3.4 Course-wide Pre and Post Course Surveys

We gave pre- and post-surveys for this course during each term that it was offered, but haven't analyzed the data yet.

3.5 Conclusion

Bibliography

- [1] Lorin W. Anderson and David R. Krathwohl, editors. A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom’s Taxonomy of Educational Objectives. Longman, New York, NY, 2001.
- [2] D.P. Bunde, A.R. Crockett, G.C. Gannod, J. Spacco, N. Thota, and C.C. Weems. Wip: Updating CS1 to a 21st-century model of computing. In Proc. Frontiers in Education (FIE), 2024.
- [3] Seymour Papert. Mindstorms: Children, Computers, and Powerful Ideas. Basic Books, New York, 1980.
- [4] L. Porter, C.B. Lee, and B. Simon. Halving fail rates using peer instruction: a study of four computer science courses. In Proc. 44th SIGCSE Technical Symp. Computer Science Education (SIGCSE TS), pages 177–182, 2013.
- [5] Leo Porter and Beth Simon. Retaining nearly one-third more majors with a trio of instructional best practices in CS1. In Proc. 44th SIGCSE Technical Symp. Computer Science Education (SIGCSE TS), pages 165–170, 2013.
- [6] M. Smith, S. Srivastava, D.P. Bunde, A. Crockett, M. Gerten, P. Maher, J. Spacco, X. Suo, J. Wang, and M. Zhu. A visual unplugged activity to introduce PDC. In Proc. Workshop on Education for High-Performance Computing (EduHPC), 2025.

Appendix

Github Link to Instructional Material

https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter3-Knox

Detailed Pre and Post Syllabus

https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter3-Knox/Detailed_Pre_and_Post_Syllabus

Organization of Material

The git repo contains the full course with the modifications described in this chapter (as taught in Fall 2025). `COURSE_CALENDAR.md` depicts the day by day presentation of material and due dates for assignments. This document has links to the actual materials, which are stored in the Archive directory. The directory `Detailed_Pre_and_Post_Syllabus` contains the course syllabus and a description of the changes made from previous offerings of the course. The materials for the three activities described in this chapter are in `Unplugged_Activities/flagmaker` (despite the name, this directory contains both the plugged and unplugged versions of the activities), `Plugged_Activities/knoxcraft` and `Plugged_Activities/greenfoot`.